# Cours sur les Algorithmes Génétiques (Genetic Algorithms)

## 1. Introduction et Définition

### Qu'est-ce qu'un Algorithme Génétique ?

Un **Algorithme Génétique (GA)** est une métaheuristique inspirée du processus de **sélection naturelle** de Darwin. Il appartient à la famille plus large des **algorithmes évolutionnaires** et de la **computation évolutionnaire**.

Les algorithmes génétiques sont utilisés pour générer des solutions de haute qualité aux problèmes d'**optimisation** et de **recherche** en utilisant des opérateurs biologiquement inspirés tels que :

- La **sélection**
- Le **croisement** (crossover)
- La **mutation**

### Historique

- **1950** : Alan Turing propose le concept de "machine apprenante" basée sur l'évolution
- **1954** : Nils Aall Barricelli effectue les premières simulations informatiques d'évolution
- **Années 1960-70** : Hans-Joachim Bremermann, Ingo Rechenberg et Hans-Paul Schwefel développent les stratégies d'évolution
- **1975** : John Holland publie "Adaptation in Natural and Artificial Systems", formalisant les algorithmes génétiques
- **Années 1980-90** : Explosion des applications pratiques et commerciales

---

## 2. Principes de Base

### Inspiration Biologique

Les algorithmes génétiques s'inspirent de l'évolution biologique :

| Concept Biologique | Équivalent en AG |
|---|---|
| Organisme/Individu | Solution candidate |
| Population | Ensemble de solutions |
| Chromosome/ADN | Représentation de la solution |
| Gène | Variable ou paramètre |
| Fitness (aptitude) | Qualité de la solution |
| Génération | Itération de l'algorithme |
| Reproduction | Création de nouvelles solutions |

| Concept Biologique | Équivalent en AG |
|---|---|
| Mutation | Modification aléatoire |
| Sélection naturelle | Sélection des meilleures solutions |

### Représentation

Les solutions sont traditionnellement représentées comme des **chaînes de bits** (0 et 1), mais d'autres encodages sont possibles :

- **Binaire** : `10110101`
- **Réels** : `[3.14, 2.71, 1.41]`
- **Permutations** : `[3, 1, 4, 2, 5]`
- **Arbres** : utilisés en programmation génétique
- **Structures complexes** : graphes, matrices, etc.

---

## 3. Comment Fonctionne un Algorithme Génétique

### Algorithme Générique

```
1. INITIALISATION
   - Créer une population initiale aléatoire de N individus

2. ÉVALUATION
   - Calculer la fitness de chaque individu

3. BOUCLE PRINCIPALE (jusqu'à condition d'arrêt) :

   a. SÉLECTION
      - Choisir les parents basés sur leur fitness

   b. CROISEMENT (Crossover)
      - Combiner les parents pour créer des enfants

   c. MUTATION
      - Appliquer des mutations aléatoires aux enfants

   d. ÉVALUATION
      - Calculer la fitness des nouveaux individus

   e. REMPLACEMENT
      - Former la nouvelle génération

   f. VÉRIFIER CONDITION D'ARRÊT
      - Nombre de générations
      - Solution satisfaisante trouvée
      - Convergence/stagnation
      - Temps écoulé
```

```
4. RETOURNER la meilleure solution
```

Pseudo-code Détaillé

```python
def algorithme_genetique(taille_pop, nb_generations, prob_croisement,
prob_mutation):
    # 1. Initialisation
    population = initialiser_population_aleatoire(taille_pop)

    for generation in range(nb_generations):
        # 2. Évaluation
        fitness = [evaluer_fitness(individu) for individu in population]

        # 3. Vérification condition d'arrêt
        if solution_satisfaisante(fitness):
            return meilleur_individu(population, fitness)

        # 4. Sélection des parents
        parents = selection(population, fitness)

        # 5. Création des enfants
        enfants = []
        for i in range(0, len(parents), 2):
            parent1, parent2 = parents[i], parents[i+1]

            # Croisement
            if random() < prob_croisement:
                enfant1, enfant2 = croisement(parent1, parent2)
            else:
                enfant1, enfant2 = parent1, parent2

            # Mutation
            if random() < prob_mutation:
                enfant1 = mutation(enfant1)
            if random() < prob_mutation:
                enfant2 = mutation(enfant2)

            enfants.extend([enfant1, enfant2])

        # 6. Remplacement
        population = former_nouvelle_generation(population, enfants)

    return meilleur_individu(population, evaluer_tous(population))
```

---

## 4. Composants Essentiels

### 4.1 Fonction de Fitness (Aptitude)

La fonction de fitness évalue la **qualité** d'une solution. Elle est **spécifique au problème** :

**Exemples :**

- **Problème du sac à dos** : maximiser la valeur totale des objets sans dépasser la capacité
- **Optimisation de fonction** : minimiser/maximiser f(x)
- **Planification** : minimiser le temps total ou le coût
- **Design** : maximiser l'efficacité ou la performance

**Caractéristiques importantes :**

- Doit être calculable pour toute solution candidate
- Doit refléter fidèlement l'objectif à optimiser
- Peut être coûteuse en calcul (problème majeur des AG)

## 4.2 Opérateurs de Sélection

La sélection détermine quels individus deviennent parents :

**a) Sélection par Roulette (Roulette Wheel)**

- Probabilité de sélection proportionnelle à la fitness
- Les meilleurs ont plus de chances, mais tous peuvent être sélectionnés

**b) Sélection par Tournoi**

- Choisir k individus aléatoirement
- Sélectionner le meilleur parmi eux
- Simple et efficace

**c) Sélection par Rang**

- Classer les individus par fitness
- Probabilité basée sur le rang, pas la valeur absolue

**d) Sélection Élitiste**

- Toujours conserver les meilleurs individus
- Garantit la non-dégradation de la meilleure solution

## 4.3 Opérateurs de Croisement (Crossover)

Le croisement combine deux parents pour créer des enfants :

**a) Croisement à Un Point**

```
Parent 1: 1 1 0 1 | 0 1 0
Parent 2: 0 1 1 0 | 1 1 1
          --------+-------
```

```
Enfant 1: 1 1 0 1 | 1 1 1
Enfant 2: 0 1 1 0 | 0 1 0
```

**b) Croisement à Deux Points**

```
Parent 1: 1 1 | 0 1 0 | 1 0
Parent 2: 0 1 | 1 0 1 | 1 1
          ----+-------+----
Enfant 1: 1 1 | 1 0 1 | 1 0
Enfant 2: 0 1 | 0 1 0 | 1 1
```

**c) Croisement Uniforme**

- Pour chaque gène, choisir aléatoirement le parent

```
Parent 1: 1 1 0 1 0 1 0
Parent 2: 0 1 1 0 1 1 1
Masque:   1 0 1 1 0 1 0
          ---------------
Enfant:   1 1 1 1 1 1 0
```

**d) Croisement Arithmétique (pour les réels)**

```
Enfant = α × Parent1 + (1-α) × Parent2
où α ∈ [0,1]
```

## 4.4 Opérateurs de Mutation

La mutation introduit de la diversité et explore de nouvelles régions :

**a) Mutation de Bit (binaire)**

```
Avant:  1 1 0 1 0 1 0
               ↓
Après:  1 1 0 0 0 1 0
```

**b) Mutation par Échange (permutations)**

```
Avant:  [3, 1, 4, 2, 5]
          ↓        ↓
```

```
Après:  [3, 5, 4, 2, 1]
```

**c) Mutation Gaussienne (réels)**

```
x_nouveau = x_ancien + N(0, σ)
où N(0, σ) est une distribution normale
```

**d) Mutation de Frontière**

- Remplacer par la limite min ou max du domaine

---

# 5. Paramètres Clés et Réglage

## Paramètres Principaux

| Paramètre | Valeurs Typiques | Impact |
|---|---|---|
| **Taille de population** | 50-500 | Plus grand = plus de diversité, mais plus lent |
| **Nombre de générations** | 100-10000 | Critère d'arrêt principal |
| **Probabilité de croisement** | 0.6-0.9 | Haut = plus d'exploration |
| **Probabilité de mutation** | 0.001-0.1 | Bas = exploitation, Haut = exploration |
| **Stratégie d'élitisme** | 1-5% meilleurs | Préserve les bonnes solutions |

## Conseils de Réglage

1. **Population trop petite** → convergence prématurée
2. **Population trop grande** → calculs coûteux
3. **Mutation trop faible** → perte de diversité (dérive génétique)
4. **Mutation trop forte** → perte des bonnes solutions
5. **Utiliser l'élitisme** pour garantir la non-dégradation

---

# 6. Applications Pratiques

## 6.1 Optimisation Numérique

- Maximisation/minimisation de fonctions complexes
- Optimisation multi-objectifs
- Espaces de recherche de grande dimension

## 6.2 Problèmes Combinatoires

- **Problème du voyageur de commerce (TSP)**
- **Problème du sac à dos**

- **Planification et ordonnancement**
- **Affectation de ressources**
- **Coloration de graphes**

### 6.3 Ingénierie et Design

- Design d'antennes (NASA ST5)
- Optimisation de formes aérodynamiques
- Conception de circuits électroniques
- Optimisation de paramètres de contrôle

### 6.4 Machine Learning

- Sélection de features
- Optimisation d'hyperparamètres
- Entraînement de réseaux de neurones (neuroévolution)
- Programmation génétique pour générer des algorithmes

### 6.5 Autres Applications

- **Finance** : optimisation de portefeuille
- **Bioinformatique** : alignement de séquences
- **Jeux** : stratégies de jeu adaptatives
- **Robotique** : planification de trajectoires
- **Art génératif** : création d'art évolutionnaire

---

## 7. Avantages des Algorithmes Génétiques

✅ **Ne nécessitent pas de gradient ou de dérivées**

- Peuvent optimiser des fonctions non-différentiables

✅ **Recherche parallèle**

- Explorent plusieurs régions simultanément

✅ **Flexibles**

- S'adaptent à tout type de problème d'optimisation

✅ **Robustes**

- Gèrent bien le bruit et les données imparfaites

✅ **Trouvent des solutions "assez bonnes"**

- Même pour des problèmes NP-difficiles

✅ **Évitent les minima locaux**

- Grâce à la diversité de la population

---

# 8. Limitations et Défis

× **Pas de garantie d'optimalité**

- Trouvent des approximations, pas forcément l'optimum global

× **Évaluation de fitness coûteuse**

- Peut nécessiter des simulations complexes
- Devient prohibitif pour des problèmes à grande échelle

× **Convergence prématurée**

- Risque de converger vers des optima locaux
- Perte de diversité génétique

× **Nombreux paramètres à régler**

- Taille de population, probabilités, etc.
- Sensible aux paramètres (problem-dependent)

× **Pas adapté à tous les problèmes**

- Problèmes simples : méthodes exactes plus efficaces
- Problèmes avec fitness binaire (pass/fail)

× **Difficile à gérer la complexité**

- Espace de recherche exponentiellement grand
- Nécessite une représentation appropriée

---

# 9. Variantes et Extensions

## 9.1 Algorithmes Génétiques Adaptatifs (AGA)

- Ajustent automatiquement les probabilités de croisement et mutation
- Maintiennent la diversité tout en convergeant

## 9.2 Algorithmes Génétiques Multi-Objectifs

- **NSGA-II** (Non-dominated Sorting Genetic Algorithm)
- **SPEA2** (Strength Pareto Evolutionary Algorithm)
- Optimisent simultanément plusieurs objectifs conflictuels

## 9.3 Algorithmes Génétiques Parallèles

- **Modèle en îles** : Plusieurs populations évoluent indépendamment avec migration
- **Modèle cellulaire** : Structure de voisinage pour la sélection

## 9.4 Algorithmes Mémétiques

- Hybridation avec des recherches locales

- Combinent évolution globale et raffinement local

### 9.5 Programmation Génétique (GP)

- Évolution de **programmes informatiques**
- Représentation en arbres d'expressions

### 9.6 Autres Algorithmes Évolutionnaires

- **Stratégies d'Évolution (ES)** : Pour l'optimisation continue
- **Programmation Évolutionnaire (EP)**
- **Évolution Différentielle (DE)**

---

## 10. Exemple Pratique : Optimisation d'une Fonction

### Problème

Maximiser la fonction : `f(x) = x * sin(10π * x) + 1.0` sur l'intervalle [-1, 2]

### Solution avec AG

```
import random
import math

# Paramètres
POPULATION_SIZE = 50
GENERATIONS = 100
MUTATION_RATE = 0.01
CROSSOVER_RATE = 0.7
GENE_LENGTH = 20  # bits pour encoder x

def decode(chromosome):
    """Convertit un chromosome binaire en valeur réelle"""
    decimal = int(''.join(map(str, chromosome)), 2)
    x = -1 + (decimal / (2**GENE_LENGTH - 1)) * 3  # [-1, 2]
    return x

def fitness(chromosome):
    """Évalue la fitness"""
    x = decode(chromosome)
    return x * math.sin(10 * math.pi * x) + 1.0

def initialize_population():
    """Crée une population initiale aléatoire"""
    return [[random.randint(0, 1) for _ in range(GENE_LENGTH)]
            for _ in range(POPULATION_SIZE)]

def selection(population, fitnesses):
    """Sélection par tournoi"""
    tournament_size = 3
    selected = []
```

```
    for _ in range(len(population)):
        contestants = random.sample(list(zip(population, fitnesses)),
tournament_size)
        winner = max(contestants, key=lambda x: x[1])[0]
        selected.append(winner[:])
    return selected

def crossover(parent1, parent2):
    """Croisement à un point"""
    if random.random() < CROSSOVER_RATE:
        point = random.randint(1, GENE_LENGTH - 1)
        child1 = parent1[:point] + parent2[point:]
        child2 = parent2[:point] + parent1[point:]
        return child1, child2
    return parent1[:], parent2[:]

def mutate(chromosome):
    """Mutation de bit"""
    for i in range(GENE_LENGTH):
        if random.random() < MUTATION_RATE:
            chromosome[i] = 1 - chromosome[i]
    return chromosome

def genetic_algorithm():
    """Algorithme génétique principal"""
    population = initialize_population()

    best_ever = None
    best_fitness_ever = float('-inf')

    for generation in range(GENERATIONS):
        # Évaluation
        fitnesses = [fitness(ind) for ind in population]

        # Suivre le meilleur
        best_idx = fitnesses.index(max(fitnesses))
        if fitnesses[best_idx] > best_fitness_ever:
            best_fitness_ever = fitnesses[best_idx]
            best_ever = population[best_idx][:]

        # Affichage du progrès
        if generation % 10 == 0:
            avg_fit = sum(fitnesses) / len(fitnesses)
            print(f"Gen {generation}: Meilleur={best_fitness_ever:.4f},
Moyen={avg_fit:.4f}")

        # Sélection
        parents = selection(population, fitnesses)

        # Croisement et Mutation
        next_generation = []
        for i in range(0, POPULATION_SIZE, 2):
            child1, child2 = crossover(parents[i], parents[i+1])
            next_generation.append(mutate(child1))
```

```
                next_generation.append(mutate(child2))

            # Élitisme : garder le meilleur
            next_generation[0] = best_ever[:]

            population = next_generation

        return best_ever, best_fitness_ever

    # Exécution
    best_solution, best_value = genetic_algorithm()
    best_x = decode(best_solution)
    print(f"\nMeilleure solution trouvée : x = {best_x:.6f}, f(x) =
    {best_value:.6f}")
```

---

# 11. Théorèmes et Concepts Importants

## 11.1 Théorème des Schémas (Schema Theorem)

- Proposé par John Holland
- Explique pourquoi les AG fonctionnent
- Les "building blocks" (schémas courts, de faible ordre, de haute fitness) se propagent exponentiellement

## 11.2 Hypothèse des Building Blocks

- Les AG construisent de bonnes solutions en combinant des "blocs de construction"
- Similaire à un enfant construisant avec des Lego

## 11.3 Théorème "No Free Lunch"

- Aucun algorithme n'est meilleur pour tous les problèmes
- Les AG doivent exploiter les connaissances du problème
- Importance de la représentation et des opérateurs adaptés

## 11.4 Convergence

- Les AG élitistes convergent (prouvé mathématiquement)
- Mais la vitesse de convergence n'est pas garantie
- Risque de convergence prématurée

---

# 12. Bonnes Pratiques

## 12.1 Choix de la Représentation

✓ Utiliser une représentation naturelle au problème ✓ Éviter les solutions invalides (ou pénaliser) ✓ Pour les réels : préférer encodage réel à binaire

## 12.2 Conception de la Fonction de Fitness

✓ Doit guider vers l'optimum ✓ Récompenser les améliorations partielles ✓ Éviter les plateaux (zones de fitness constante) ✓ Pénaliser les contraintes violées progressivement

### 12.3 Gestion de la Diversité

✓ Utiliser une population suffisamment grande ✓ Ajuster les taux de mutation ✓ Implémenter du "niching" ou "crowding" ✓ Techniques de partage de fitness

### 12.4 Critères d'Arrêt

✓ Nombre maximum de générations ✓ Fitness cible atteinte ✓ Stagnation (pas d'amélioration sur N générations) ✓ Budget de calcul épuisé

### 12.5 Optimisation des Performances

✓ Paralléliser les évaluations de fitness ✓ Utiliser des approximations de fitness si coûteuses ✓ Mémoriser les solutions déjà évaluées (cache) ✓ Considérer des algorithmes hybrides

---

## 13. Comparaison avec Autres Méthodes

| Méthode | Avantages | Inconvénients |
|---|---|---|
| **Algorithmes Génétiques** | Pas de gradient nécessaire, parallèle, flexible | Lent, beaucoup de paramètres |
| **Gradient Descent** | Rapide, garanti (convexe) | Nécessite gradient, minima locaux |
| **Recuit Simulé** | Simple, un seul individu | Plus lent que AG, séquentiel |
| **PSO (Particle Swarm)** | Moins de paramètres, rapide | Convergence prématurée |
| **Branch & Bound** | Solution exacte | Exponentiel, impraticable |
| **Hill Climbing** | Très simple | Minima locaux |

---

## 14. Ressources et Outils

### 14.1 Bibliothèques Python

- **DEAP** (Distributed Evolutionary Algorithms in Python)
- **PyGAD** (Python Genetic Algorithm)
- **Genetica**
- **Scikit-opt**

### 14.2 Autres Langages

- **MATLAB** : Global Optimization Toolbox (ga function)
- **Java** : JGAP, Jenetics
- **C++** : GAlib, EO (Evolving Objects)
- **R** : GA package

### 14.3 Livres Recommandés

- "Genetic Algorithms in Search, Optimization and Machine Learning" - David Goldberg
- "An Introduction to Genetic Algorithms" - Melanie Mitchell
- "Adaptation in Natural and Artificial Systems" - John Holland

### 14.4 Applications Célèbres

- **Antenne NASA ST5** : Design d'antenne évoluée
- **Evolver** : Premier produit commercial GA (1989)
- **Optimisation de portefeuilles financiers**
- **Ordonnancement de production**

---

## 15. Conclusion

Les **Algorithmes Génétiques** sont des outils puissants pour résoudre des problèmes d'optimisation complexes où les méthodes traditionnelles échouent. Leur force réside dans :

🔹 **Polyvalence** : s'appliquent à quasi tout problème 🔹 **Robustesse** : gèrent bien le bruit et l'incertitude 🔹 **Exploration globale** : évitent les pièges des optima locaux 🔹 **Parallélisme naturel** : exploitent les architectures modernes

Cependant, ils requièrent :

- Une bonne compréhension du problème
- Un réglage approprié des paramètres
- Une patience pour les évaluations coûteuses
- Une acceptation de solutions approximatives

**Quand utiliser les AG ?** ✅ Problèmes NP-difficiles ✅ Fonctions non-différentiables ou discontinues ✅ Espaces de recherche complexes et multimodaux ✅ Optimisation multi-objectifs ✅ Quand une bonne solution suffit (pas besoin d'optimum)

**Quand NE PAS utiliser les AG ?** ✗ Problèmes avec solutions exactes efficaces ✗ Fonctions convexes simples (gradient descent mieux) ✗ Si évaluation de fitness extrêmement coûteuse ✗ Si besoin d'optimalité garantie

Les AG restent un domaine de recherche actif avec de nouvelles variantes et applications apparaissant régulièrement. Leur simplicité conceptuelle et leur efficacité pratique en font un outil essentiel dans la boîte à outils de tout data scientist ou ingénieur en optimisation.

---

## Annexe : Glossaire

- **Chromosome** : Représentation codée d'une solution
- **Gène** : Un élément du chromosome (un bit, un paramètre)
- **Allèle** : Valeur d'un gène
- **Fitness** : Qualité/aptitude d'une solution
- **Génération** : Une itération de l'algorithme
- **Population** : Ensemble de solutions candidates

- **Parent** : Solution sélectionnée pour la reproduction
- **Offspring** : Nouvelle solution créée
- **Convergence** : État où la population devient homogène
- **Élitisme** : Conservation des meilleures solutions
- **Drift génétique** : Perte de diversité par hasard
- **Pressure de sélection** : Force de sélection des meilleurs
- **Landscape de fitness** : Visualisation de l'espace de recherche
- **Exploration** : Recherche de nouvelles régions
- **Exploitation** : Raffinement des solutions connues

---

**Fin du Cours**

*Ce document fournit une base solide pour comprendre et appliquer les Algorithmes Génétiques. Pour aller plus loin, pratiquez sur des problèmes réels et étudiez les variantes avancées selon vos besoins spécifiques.*